



# Climb Phase Documentation

**Scope:** Complete documentation of the climb phase simulation system, including strategy-based climb trajectories, 3D dynamic programming optimization, penalty systems, and integration with mission analysis for optimal fuel consumption.

## Table of Contents

1. [System Overview and Objectives](#)
2. [Mathematical Foundation](#)
3. [System Architecture and Data Structures](#)
4. [Strategy System Implementation](#)
5. [Dynamic Programming Optimization](#)
6. [Penalty System and Guidance](#)
7. [Code Execution Flow and Logic](#)
8. [Integration and Interface](#)
9. [Validation and Quality Assurance](#)

## 1) System Overview and Objectives

### 1.1 Purpose and Scope

The climb phase simulation implements a comprehensive climb trajectory analysis system that models aircraft performance during the climb segment of flight. The system provides multiple climb strategies, 3D dynamic programming optimization, and penalty-based guidance for realistic climb trajectory simulation.

### 1.2 System Objectives

**Primary Objectives:**

- **Fuel-Optimal Climb:** Find minimum fuel consumption climb trajectories
- **Strategy Analysis:** Compare different climb strategies and their performance
- **Realistic Trajectories:** Generate physically realizable climb paths
- **Mission Integration:** Provide seamless connection with cruise and descent phases

#### Key Components:

- Multiple climb strategies (linear, exponential, constant speed/Mach)
- 3D dynamic programming optimization (Mach × Altitude × Lever)
- Penalty-based guidance system for trajectory realism
- Engine performance integration with FADEC-like control
- Atmospheric property calculations using ISA model
- Integration with aerodynamics tables from Excel

## 1.3 System Flow Overview

The climb simulation follows a logical progression:

1. **Initialization:** Load aerodynamics data, engine model, and system parameters
  2. **Strategy Simulation:** Execute various climb strategies for comparison
  3. **3D Optimization:** Run dynamic programming optimization for fuel-optimal path
  4. **Penalty Integration:** Apply guidance penalties for realistic trajectories
  5. **Results Analysis:** Generate comprehensive performance data and visualizations
  6. **Mission Integration:** Provide climb results for cruise phase initialization
- 

## 2) Mathematical Foundation

### 2.1 Climb Flight Physics

**Theory:** Climb flight requires excess thrust beyond that needed for level flight. The specific excess power ( $P_s$ ) determines the aircraft's climb capability and fuel consumption rate.

**Mathematical Formulation:**

$$P_s = \frac{(T_{total} - D) \times V_{TAS}}{W}$$

$$\dot{h} = P_s \times \sin(\gamma) \approx P_s \times \frac{\dot{h}}{V_{TAS}}$$

$$J = \frac{\dot{m}_{fuel}}{P_s} \quad [kg/m]$$

Where:

- $P_s$  = Specific excess power (m/s)
- $T_{\{total\}}$  = Total thrust (N)
- $D$  = Total drag (N)
- $V_{\{TAS\}}$  = True airspeed (m/s)
- $W$  = Aircraft weight (N)
- $J$  = Fuel cost density (kg/m)

## 2.2 Dynamic Programming Formulation

**Theory:** The climb optimization problem can be formulated as a discrete-time dynamic programming problem using Bellman's principle of optimality.

**Continuous Problem Formulation:**

$$\min_{\ell(\cdot), M(\cdot)} J_{tot} = \int_{h_0}^{h_f} \underbrace{\frac{\dot{m}(\ell, M, h)}{P_s(\ell, M, h)}}_{J(\ell, M, h) \text{ [kg/m]}} dh$$

**Discrete State Space:**

$$S = \{M_i\} \times \{h_j\} \times \{\ell_k\}$$

**Bellman Equation:**

$$J^*(M_i, h_j, \ell_k) = \min_{\ell'} [J(M_i, h_j, \ell_k, \ell') + \gamma J^*(M_{i'}, h_{j'}, \ell')]$$

## 2.3 Strategy Energy Allocation

**Theory:** Climb strategies allocate energy between climb rate and speed optimization using configurable energy fractions.

**Energy Allocation Model:**

$$E_{total} = E_{climb} + E_{speed}$$

$$\dot{h} = cw \times P_s$$

$$\dot{V} = sw \times \frac{P_s \times W}{V \times m}$$

Where:

- `cw` = Climb weight fraction
- `sw` = Speed weight fraction
- `cw + sw = 1.0`

## 3) System Architecture and Data Structures

### 3.1 Core System Parameters

#### Aircraft and Engine Parameters:

|                                        |                            |
|----------------------------------------|----------------------------|
| <code>N_ENGINES = 2</code>             | # Number of engines        |
| <code>INITIAL_MASS_KG = 65000.0</code> | # Fixed aircraft mass (kg) |
| <code>S_REF_M2 = 122.4</code>          | # Reference wing area (m²) |

#### Engine Operational Limits:

|                                  |                                    |
|----------------------------------|------------------------------------|
| <code>M_MIN_DEFAULT = 0.0</code> | # Default minimum operational Mach |
| <code>M_MMO = 0.94</code>        | # Maximum operating Mach           |
| <code>CL_MAX = None</code>       | # Maximum lift coefficient         |

#### Grid and Discretization Parameters:

|                                     |                                              |
|-------------------------------------|----------------------------------------------|
| <code>TARGET_ALT_M = 10000.0</code> | # Target altitude for 3D DP optimization (m) |
| <code>ALT_STEP_M = 200.0</code>     | # Altitude step size (m)                     |
| <code>MACH_COLS = 81</code>         | # Number of Mach discretization points       |
| <code>N_PLOT_STEPS = 50</code>      | # Number of trajectory points for plotting   |

## Strategy Energy Management:

```
E_DOT_CMD = 14 # Command energy rate (m/s)
```

## 3.2 Data Structures

### 3.2.1 StrategyRun

**Purpose:** Complete results container for strategy-based climb simulation.

**Structure:**

```
@dataclass
class StrategyRun:
    label: str # Strategy identifier
    h_m: np.ndarray # Altitude trajectory
    V_ms: np.ndarray # Velocity trajectory
    t_s: np.ndarray # Time progression
    lever: np.ndarray # Engine lever positions
    Ttot_N: np.ndarray # Total thrust
    D_N: np.ndarray # Total drag
    Ps_ms: np.ndarray # Specific excess power
    mdot_kgps: np.ndarray # Fuel flow rate
    dt_s: np.ndarray # Time steps
    dFuel_kg: np.ndarray # Fuel consumed per step
    cumFuel_kg: np.ndarray # Cumulative fuel consumption
```

### 3.2.2 MinFuelSchedule

**Purpose:** Results container for 3D dynamic programming optimization.

**Structure:**

```

@dataclass
class MinFuelSchedule:
    alt_m: np.ndarray          # Optimal altitude trajectory
    mach: np.ndarray           # Optimal Mach trajectory
    lever: np.ndarray           # Optimal lever trajectory
    cumFuel_kg: np.ndarray      # Cumulative fuel consumption
    dt_s: np.ndarray           # Time steps
    total_time_s: float         # Total climb time
    total_fuel_kg: float        # Total fuel consumed

```

## 3.3 System Integration Components

### Aerodynamics System:

```

class AeroTables:
    def get_drag(self, mach: float, altitude_m: float) -> float
    def get_lift_coefficient(self, mach: float, altitude_m: float) -> float

```

### Engine System:

```

class EngineWrapper:
    def thrust_with_lever(self, lever: float, mach: float, altitude_m: float) -> float
    def tsfc_current(self) -> float

```

### Atmosphere System:

```

def isa_properties(h_m: float) -> Tuple[float, float, float]
def a_from_altitude(h_m: float) -> float

```

---

## 4) Strategy System Implementation

### 4.1 Strategy Types and Energy Allocation

#### Linear Strategies:

```
@staticmethod
def linear_climb(altitude, velocity, altitude_fraction):
    """Linear climb strategy with constant energy allocation."""
    climb_norm = altitude_fraction # e.g., 0.10 = 10% climb
    speed_norm = 1.0 - climb_norm # 90% speed
    return climb_norm, speed_norm
```

#### Exponential Strategies:

```
@staticmethod
def increasing_climb(altitude, velocity, altitude_fraction):
    """Exponential climb strategy with increasing climb emphasis."""
    af_param, current_af = altitude_fraction
    min_climb = 0.1
    max_climb = 0.9
    exponent = 1.0 + 2.0 * af_param
    exp_factor = current_af ** exponent
    climb_norm = min_climb + (max_climb - min_climb) * exp_factor
    speed_norm = 1.0 - climb_norm
    return climb_norm, speed_norm
```

#### Constant Strategies:

```
@staticmethod
def constant_mach(altitude, velocity, altitude_fraction):
    """Constant Mach strategy maintaining fixed Mach number."""
    return 1.0, 0.0 # 100% climb, 0% speed change
```

### 4.2 Strategy Simulation Engine

**Function:** `ClimbingCore.StrategyManager.simulate_strategy_path()`

## Simulation Flow:

```
@staticmethod
def simulate_strategy_path(*, label: str, aero: AeroTables, eng: EngineWrapper,
                           mass0_kg: float, h0_m: float, V0_ms: float,
                           target_alt_m: float, dt: float,
                           strategy_fn: Callable, altitude_fraction: Optional[float]) -> StrategyRun:

    # Initialize trajectory arrays
    h_hist, V_hist, t_hist = [float(h0_m)], [float(V0_ms)], [0.0]
    lever_hist, Ttot_hist, D_hist, Ps_hist = [], [], [], []
    mdot_hist, dt_hist, dFuel_hist, cumFuel_hist = [], [], [], [0.0]

    mass_kg = float(mass0_kg) # Fixed mass for strategies

    # Main simulation loop
    while h_hist[-1] < target_alt_m:
        h = float(h_hist[-1])
        V = float(V_hist[-1])
        t = float(t_hist[-1])

        # Calculate current conditions
        a = a_from_altitude(h)
        M = V / max(a, 1e-9)
        Mq = float(np.clip(M, M_MIN_DEFAULT, 0.94))

        # Apply strategy energy allocation
        current_altitude_fraction = (h - h0_m) / (target_alt_m - h0_m)
        cw, sw = strategy_fn(h, V, altitude_fraction)

        # Calculate energy allocation
        climb_weight = cw
        speed_weight = sw

    # Continue simulation...
```

## 4.3 Strategy Set Configuration

### Strategy Set Builder:



```

@staticmethod
def build_strategy_set() -> List[Tuple[str, Callable, Optional[float]]]:
    """Build comprehensive set of climb strategies for comparison."""
    out = []
    for af in [0.10, 0.20, 0.30]:
        out.append((f"Linear AF={af:.2f}",
                    ClimbingCore.StrategyManager.StrategyProfiles.FixedEnergy.Linear.profile, af))
    for af in [0.50]:
        out.append((f"Exp climb AF={af:.2f}",
                    ClimbingCore.StrategyManager.StrategyProfiles.FixedEnergy.Exponential.increasing, af))
        out.append((f"Exp speed AF={af:.2f}",
                    ClimbingCore.StrategyManager.StrategyProfiles.FixedEnergy.Exponential.increasing, af))
    out.append(("Constant speed",
                ClimbingCore.StrategyManager.StrategyProfiles.ConstantRates.constant_speed, None))
    out.append(("Constant Mach",
                ClimbingCore.StrategyManager.StrategyProfiles.ConstantRates.constant_mach(), None))
    return out

```

## 5) Dynamic Programming Optimization

### 5.1 3D State Space Definition

#### State Variables:

- **Mach Number:**  $M \in [M_{\min}, M_{\text{MMO}}]$
- **Altitude:**  $h \in [h_0, h_{\text{target}}]$
- **Engine Lever:**  $\ell \in [0.0, 1.0]$

#### Discretization:

```
# Mach grid
M_grid = np.linspace(M_MIN_DEFAULT, M_MMO, MACH_COLS)

# Altitude grid
H_sched = np.arange(h0_m, target_alt_m + ALT_STEP_M, ALT_STEP_M)

# Lever grid
lever_samples = 50
lever_grid = np.linspace(0.0, 1.0, lever_samples)
```

## 5.2 Bellman's Principle Implementation

**Function:** `ClimbingCore.DynamicProgrammingOptimizer.solve_3d_fixed_mass()`

**Algorithm:**

```

@staticmethod
def solve_3d_fixed_mass(aero: AeroTables, eng: EngineWrapper,
                        M_grid: np.ndarray, H_sched: np.ndarray,
                        lever_samples: int = 10,
                        target_mach: float = None,
                        target_mach_tolerance: float = 0.02,
                        start_mach: float = None,
                        start_lever: float = None) -> MinFuelSchedule:

    # Initialize 3D cost grid
    J_grid_3d = np.full((len(H_sched), len(M_grid), lever_samples), np.inf)

    # Set terminal condition
    J_grid_3d[0, :, :] = 0.0 # At initial altitude

    # Forward pass - Bellman's equation
    for k in range(1, len(H_sched)): # For each altitude level
        h_current = H_sched[k]
        h_previous = H_sched[k-1]
        dh = h_current - h_previous

        for i, M in enumerate(M_grid): # For each Mach
            for j, lever in enumerate(lever_grid): # For each lever

                # Calculate cost for this state
                cost = compute_3d_cost(aero, eng, M, h_current, lever, mass0_kg)

                # Find optimal transition from previous altitude
                min_cost_prev = np.inf
                for i_prev in range(len(M_grid)):
                    for j_prev in range(lever_samples):
                        if J_grid_3d[k-1, i_prev, j_prev] < np.inf:
                            transition_cost = cost * dh
                            total_cost = J_grid_3d[k-1, i_prev, j_prev] + transition_cost
                            min_cost_prev = min(min_cost_prev, total_cost)

                J_grid_3d[k, i, j] = min_cost_prev

    # Backtracking to find optimal path

```

```
return backtrack_optimal_path(J_grid_3d, M_grid, H_sched, lever_grid)
```

## 5.3 Cost Function Calculation

**Function:** `ClimbingCore.DynamicProgrammingOptimizer.compute_3d_cost()`

**Cost Calculation:**

```
@staticmethod
def compute_3d_cost(aero: AeroTables, eng: EngineWrapper,
                    mach: float, altitude_m: float, lever: float,
                    mass_kg: float) -> float:

    # Calculate atmospheric properties
    T, p, rho = isa_properties(altitude_m)
    a = a_from_altitude(altitude_m)
    V_tas = mach * a

    # Calculate thrust and drag
    thrust_per_engine = eng.thrust_with_lever(lever, mach, altitude_m)
    thrust_total = N_ENGINES * thrust_per_engine if thrust_per_engine else 0.0
    drag = aero.get_drag(mach, altitude_m)

    # Calculate specific excess power
    weight = mass_kg * G_C
    Ps = (thrust_total - drag) * V_tas / weight if weight > 0 else 0.0

    # Calculate fuel flow
    tsfc = eng.tsfc_current()
    fuel_flow = thrust_total * tsfc if tsfc > 0 else 0.0

    # Calculate cost density J = fuel_flow / Ps
    if Ps > 0:
        J = fuel_flow / Ps # kg/m
    else:
        J = np.inf # Infeasible state

    return J
```

---

## 6) Penalty System and Guidance

### 6.1 Mach Trajectory Guidance

**Purpose:** Guide optimization toward realistic Mach trajectories that follow typical climb profiles.

**Penalty Function:**

```
def calculate_mach_penalty(mach: float, altitude_m: float,
                          target_alt_m: float) -> float:
    """Calculate Mach trajectory guidance penalty."""

    altitude_fraction = altitude_m / target_alt_m

    # Typical climb Mach profile: start low, increase with altitude
    target_mach = 0.5 + 0.3 * altitude_fraction # 0.5 to 0.8 Mach

    # Penalty for deviation from target
    mach_deviation = abs(mach - target_mach)
    penalty_weight = MACH_PENALTY_BASE_WEIGHT

    # Increase penalty weight with altitude (urgency)
    urgency_multiplier = 1.0 + URGENCY_MULTIPLIER * altitude_fraction
    penalty_weight *= urgency_multiplier

    return penalty_weight * (mach_deviation ** 2)
```

### 6.2 Lever Position Penalties

**Purpose:** Penalize excessive lever usage to maintain realistic engine operation.

**Penalty Function:**

```

def calculate_lever_penalty(lever: float) -> float:
    """Calculate lever position penalty for excessive usage."""

    if lever <= LEVER_PENALTY_THRESHOLD:
        return 0.0 # No penalty below threshold

    # Cubic penalty growth above threshold
    excess = lever - LEVER_PENALTY_THRESHOLD
    penalty = LEVER_PENALTY_WEIGHT * (excess ** LEVER_PENALTY_EXPONENT)

    # Critical range penalties
    if lever >= LEVER_PENALTY_CRITICAL_THRESHOLD:
        penalty *= LEVER_PENALTY_CRITICAL_MULTIPLIER
    if lever >= LEVER_PENALTY_ULTRA_CRITICAL_THRESHOLD:
        penalty *= LEVER_PENALTY_ULTRA_CRITICAL_MULTIPLIER

    return penalty

```

## 6.3 Integrated Cost Function

### Total Cost Calculation:

```

def calculate_total_cost(aero: AeroTables, eng: EngineWrapper,
                        mach: float, altitude_m: float, lever: float,
                        mass_kg: float, target_alt_m: float) -> float:

    # Base fuel cost
    base_cost = compute_3d_cost(aero, eng, mach, altitude_m, lever, mass_kg)

    # Add penalties
    mach_penalty = calculate_mach_penalty(mach, altitude_m, target_alt_m)
    lever_penalty = calculate_lever_penalty(lever)

    total_cost = base_cost + mach_penalty + lever_penalty

    return total_cost

```

## 7.1 System Entry Point and Initialization

## 7.2 Strategy Simulation Flow

### Strategy Execution:

```
# 2. Strategy Simulation
def run_strategy_simulation():
    strategies = ClimbingCore.StrategyManager.build_strategy_set()
    strategy_results = []

    DT_S = 0.2 # Time step for strategy simulation

    for name, fn, af in strategies:
        # Simulate each strategy
        sr = ClimbingCore.StrategyManager.simulate_strategy_path(
            label=name,
            aero=aero,
            eng=eng,
            mass0_kg=INITIAL_MASS_KG,
            h0_m=10.0, # Start altitude
            V0_ms=85.0, # Start velocity
            target_alt_m=TARGET_ALT_M,
            dt=DT_S,
            strategy_fn=fn,
            altitude_fraction=af
        )

        # Resample to common grid for comparison
        sr_resampled = ClimbingCore.StrategyManager.resample_strategy_run(sr, N_PLOT_STEPS)
        strategy_results.append(sr_resampled)

    return strategy_results
```

## 7.3 Dynamic Programming Flow

### 3D Optimization:



```

# 3. 3D Dynamic Programming Optimization
def run_3d_optimization():
    # Set up grids
    M_min, M_max = float(aero.mach_grid[0]), float(aero.mach_grid[-1])
    M_grid = np.linspace(M_min, M_max, MACH_COLS)

    # Create uniform altitude steps
    uniform_step_size = TARGET_ALT_M / N_PLOT_STEPS # 200m steps
    H_sched = np.arange(10.0, TARGET_ALT_M + uniform_step_size, uniform_step_size)

    # Calculate starting Mach from V0_ms=85.0 m/s at 10m altitude
    a = atmospheric_props.a_from_altitude(10.0)
    start_mach = 85.0 / a # Starting Mach from V0_ms=85.0 m/s

    # Run 3D DP optimization
    dp_sched, dp_info = ClimbingCore.DynamicProgrammingOptimizer.solve_3d_fixed_mass(
        aero=aero,
        eng=eng,
        M_grid=M_grid,
        H_sched=H_sched,
        lever_samples=50,
        target_mach=0.7,
        target_mach_tolerance=0.015,
        start_mach=start_mach,
        start_lever=0.85
    )

    return dp_sched, dp_info

```

## 7.4 Visualization and Analysis Flow

### Results Processing:

```

# 4. Results Analysis and Visualization
def analyze_results(strategy_results, dp_result):
    # Convert DP result to StrategyRun format for plotting
    dp_strategy_run = convert_dp_to_strategy_run(dp_result, N_PLOT_STEPS)

    # Combine all results for comparison
    all_results = strategy_results + [dp_strategy_run]

    # Generate interactive plots
    plot_strategies_interactive(all_results, Ps_base, M_grid, H_plot)
    plot_J_3d_plotly(M_grid, H_sched, lever_grid, J_grid_3d, min_path=dp_result)
    create_strategy_comparison_plots(all_results)

    return all_results

```

## 7.5 Cruise Integration Flow

### Cruise Simulation Integration:

```

# 5. Cruise Simulation using climb results
def run_cruise_simulation_from_climb():
    # Use optimal climb result for cruise initialization
    cruise_results = run_cruise_simulation(
        climb_result=dp_sched, # Use DP optimal result
        initial_mass_kg=INITIAL_MASS_KG,
        target_distance_km=1000.0, # 1000 km cruise
        aero=aero,
        engine=eng,
        time_step_s=60.0, # 1 minute steps
        create_plots=True
    )

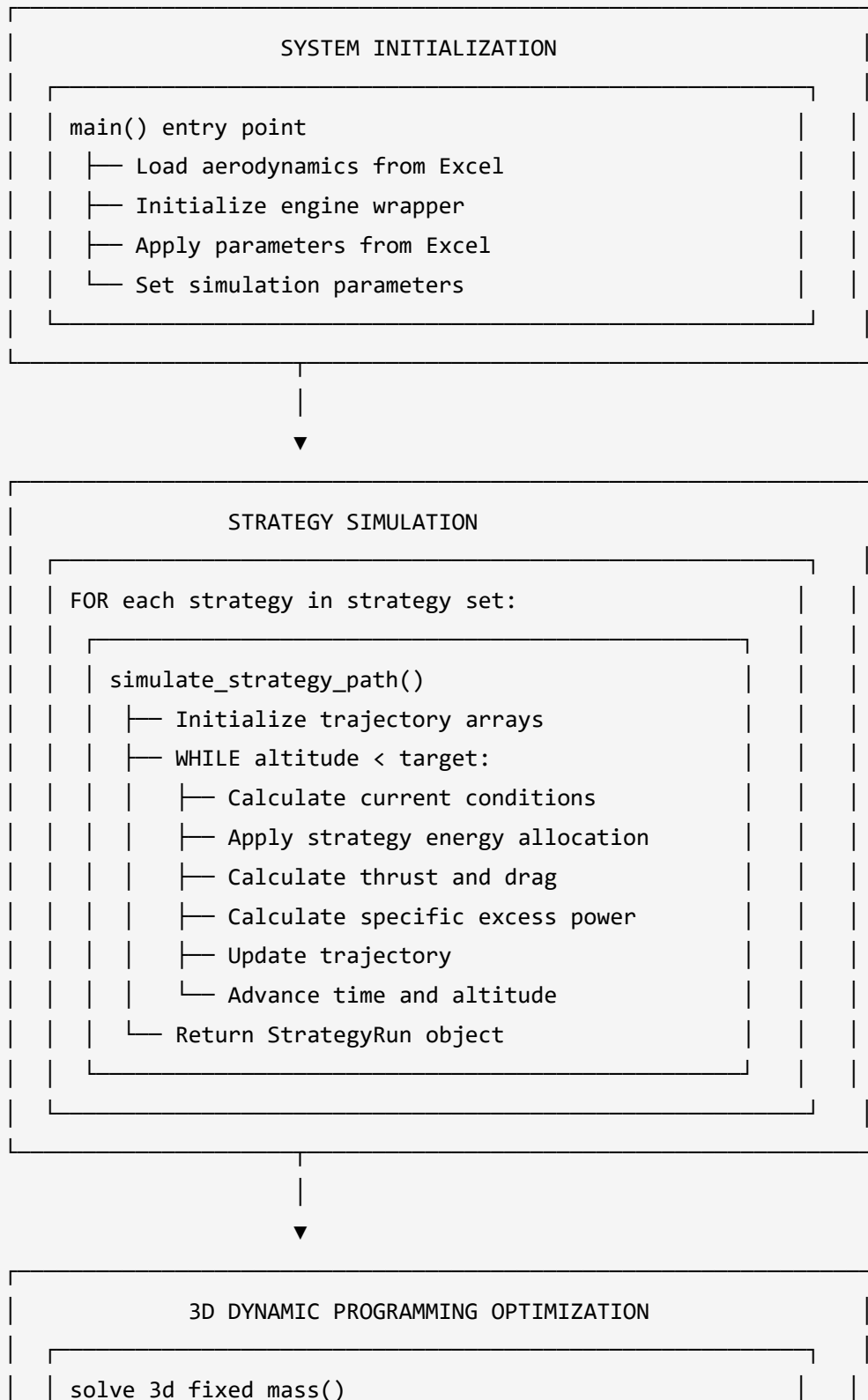
    return cruise_results

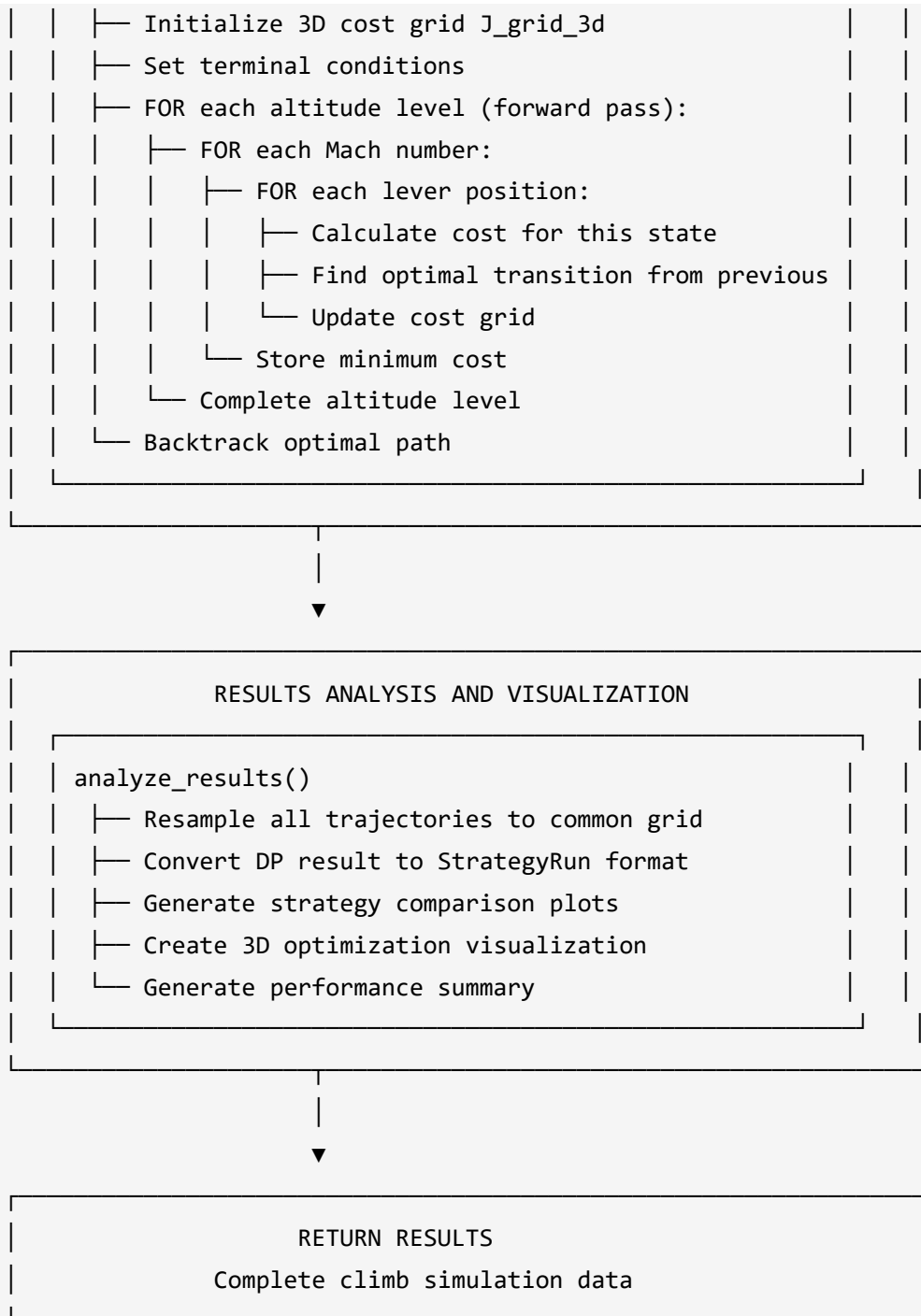
```

## 7.6 Complete Execution Flow

## CLIMB SIMULATION EXECUTION FLOW

=====





## 7.6 Function Call Hierarchy

```

main()
├─ AeroTables(AERO_XLSX, AERO_SHEET)
│   ├─ load_excel_data()
│   ├─ apply_parameters_to_globals()
│   └─ validate_data()
│
├─ EngineWrapper(ENGINE_STUB_PATH)
│   ├─ initialize_engine_model()
│   └─ setup_caching()
│
├─ run_strategy_simulation()
│   ├─ StrategyManager.build_strategy_set()
│   └─ FOR each strategy:
│       └─ StrategyManager.simulate_strategy_path()
│           ├─ initialize_trajectory_arrays()
│           └─ WHILE altitude < target:
│               │   ├─ calculate_current_conditions()
│               │   ├─ apply_strategy_energy_allocation()
│               │   ├─ calculate_thrust_and_drag()
│               │   ├─ calculate_specific_excess_power()
│               │   └─ update_trajectory()
│               └─ └─ advance_time_and_altitude()
│                   └─ return_StrategyRun()
│
├─ run_3d_optimization()
│   └─ DynamicProgrammingOptimizer.solve_3d_fixed_mass()
│       ├─ initialize_3d_cost_grid()
│       ├─ set_terminal_conditions()
│       └─ FOR each altitude level:
│           │   └─ FOR each Mach:
│           │       │   └─ FOR each lever:
│           │       │       │   └─ compute_3d_cost()
│           │       │       │       │   └─ calculate_atmospheric_properties()
│           │       │       │       │   └─ calculate_thrust_and_drag()
│           │       │       │       │   └─ calculate_specific_excess_power()
│           │       │       │       └─ └─ calculate_fuel_cost_density()
│           │       │       └─ └─ find_optimal_transition()
│           │       └─ └─ update_cost_grid()
│           └─ └─ store_minimum_cost()

```

```

|         |   └─ complete_altitude_level()
|         └─ backtrack_optimal_path()
|
└─ analyze_results()
    └─ resample_trajectories()
    └─ convert_dp_to_strategy_run()
    └─ plot_strategy_comparison()
    └─ plot_3d_optimization()

```

## 8) Integration and Interface

### 8.1 Mission Analysis Integration

**Function:** `extract_climb_results_for_cruise()`

**Integration Process:**

```

def extract_climb_results_for_cruise(dp_result: MinFuelSchedule) -> Dict[str, Any]:
    """Extract climb results for cruise phase initialization."""

    # Get final state from optimal climb
    final_altitude = float(dp_result.alt_m[-1])
    final_mach = float(dp_result.mach[-1])
    fuel_consumed_climb = float(dp_result.cumFuel_kg[-1])
    climb_time = float(dp_result.total_time_s)

    # Calculate current weight after climb
    current_weight = INITIAL_MASS_KG - fuel_consumed_climb

    return {
        'final_altitude_m': final_altitude,
        'final_mach': final_mach,
        'fuel_consumed_climb_kg': fuel_consumed_climb,
        'climb_time_s': climb_time,
        'current_weight_kg': current_weight
    }

```



## 8.2 Main Interface Functions

### Strategy Analysis Interface:

```
def run_climb_analysis(aero_xlsx: str, engine_stub_path: str,
                      initial_mass_kg: float = INITIAL_MASS_KG,
                      target_alt_m: float = TARGET_ALT_M,
                      create_plots: bool = True) -> Dict[str, Any]:
    """Main interface for complete climb analysis."""

    # Initialize systems
    aero = AeroTables(aero_xlsx, AERO_SHEET)
    eng = EngineWrapper(engine_stub_path)

    # Run strategy simulations
    strategy_results = run_strategy_simulation(aero, eng, initial_mass_kg, target_alt_m)

    # Run 3D optimization
    dp_result = run_3d_optimization(aero, eng, target_alt_m)

    # Analyze and visualize results
    if create_plots:
        analyze_results(strategy_results, dp_result)

    # Prepare results for mission integration
    cruise_inputs = extract_climb_results_for_cruise(dp_result)

    return {
        'strategy_results': strategy_results,
        'optimal_climb': dp_result,
        'cruise_inputs': cruise_inputs
    }
```

### Backward Compatibility Functions:

```
# Standalone functions for backward compatibility
def simulate_strategy_path(*, label: str, aero: AeroTables, eng: EngineWrapper,
                           mass0_kg: float, h0_m: float, V0_ms: float,
                           target_alt_m: float, dt: float,
                           strategy_fn: Callable, altitude_fraction: Optional[float]) -> StrategyRun:
    """Backward compatibility wrapper for ClimbingCore.simulate_strategy_path"""
    return ClimbingCore.StrategyManager.simulate_strategy_path(
        label=label, aero=aero, eng=eng, mass0_kg=mass0_kg, h0_m=h0_m, V0_ms=V0_ms,
        target_alt_m=target_alt_m, dt=dt, strategy_fn=strategy_fn, altitude_fraction=altitude_fraction
    )

def resample_strategy_run(sr: StrategyRun, n_samples: int) -> StrategyRun:
    """Backward compatibility wrapper for ClimbingCore.resample_strategy_run"""
    return ClimbingCore.StrategyManager.resample_strategy_run(sr, n_samples)
```

## 8.3 Performance Metrics

### Key Performance Indicators:

- **Fuel Consumption:** Total fuel burned during climb
  - **Climb Time:** Time required to reach target altitude
  - **Average Climb Rate:** Mean vertical velocity
  - **Energy Efficiency:** Fuel consumed per unit altitude gained
  - **Engine Utilization:** Average lever position and thrust usage
-